

HPL 2026

WEEK 1 — BACKEND

Technical Design & Architecture Template

HPL 2026 • SMVITM Bantakal • in association with Code Troopers

TEAM	PROJECT	TRACK	WEEK
[Team name]	[Project name]	Backend	1

Purpose of this document

This is the team's Week 1 backend design. It should make the architecture, data model, business logic, APIs, security boundaries, and important backend processes easy for a judge to understand and evaluate.

Judge-first principle

Prefer diagrams, tables, concise decisions, and concrete examples over generic architecture prose. A judge should be able to understand the system quickly and trace an important requirement from workflow → business rule → API → database.

1. System Overview

Give a concise overview of what the backend is responsible for. Do not write a general project introduction.

Item	Details
Backend Responsibility	[What does this backend own and do?]
Primary actors / roles	[e.g. Senior Citizen, Volunteer, Police Admin]
Functional Decomposition	[What logical modules make up the backend?]
Main entities	[List the important domain entities]

2. Backend Architecture

Insert one clear architecture diagram. Show clients, backend components, database, queues/workers, and external services where applicable.

Architecture diagram

Use a single diagram that shows how major components communicate. Avoid detailed class diagrams here.

[INSERT ARCHITECTURE DIAGRAM HERE]

Components

Component	Responsibility	Technology
API / Server	[What does this component do?]	[Framework / runtime]
Database	[What data does it own?]	[DB]
Background worker	[If applicable]	[Technology]
Cache / queue	[If applicable]	[Technology]
External service	[If applicable]	[Service]

Key Architecture Decisions

Only document meaningful decisions. Avoid generic claims such as 'scalable', 'secure', or 'industry standard' without explaining the concrete choice.

ID	Decision	Reason	Trade-off / Limitation
AD-01	[e.g. Modular monolith]	[Why was it chosen?]	[What does it sacrifice?]
AD-02	[...]	[...]	[...]
AD-03	[...]	[...]	[...]

3. Core Backend Workflows

Show the 3–5 most important end-to-end backend workflows. Use flow diagrams. Focus on what the backend does, not UI screens.

[INSERT WORKFLOW DIAGRAMS HERE]

4. Data Model

Insert an ER diagram showing the important entities and their relationships. Keep it readable; detailed columns are documented below.

[INSERT ER DIAGRAM HERE]

Database Schema

Define every table exactly once. If multiple modules use the same table, define it here once and reference it elsewhere.

TABLE: [table_name]

Column	Type	PK / FK	Nullable	Default	Description
id	[UUID / INT]	PK	No	[e.g. generated]	[Description]

[field]	[type]	[PK / FK / —]	[Yes / No]	[value / —]	[Description]
[field]	[type]	[PK / FK / —]	[Yes / No]	[value / —]	[Description]
created_at	[TIMESTAMP]	—	No	[...]	[Description]

[Add/remove rows as required.]

TABLE: [table_name]

Column	Type	PK / FK	Nullable	Default	Description
id	[UUID / INT]	PK	No	[e.g. generated]	[Description]
[field]	[type]	[PK / FK / —]	[Yes / No]	[value / —]	[Description]
[field]	[type]	[PK / FK / —]	[Yes / No]	[value / —]	[Description]
created_at	[TIMESTAMP]	—	No	[...]	[Description]

[Add/remove rows as required.]

TABLE: [table_name]

Column	Type	PK / FK	Nullable	Default	Description
id	[UUID / INT]	PK	No	[e.g. generated]	[Description]
[field]	[type]	[PK / FK / —]	[Yes / No]	[value / —]	[Description]
[field]	[type]	[PK / FK / —]	[Yes / No]	[value / —]	[Description]
created_at	[TIMESTAMP]	—	No	[...]	[Description]

[Add/remove rows as required.]

5. Business Logic & Rules

Document rules that determine what the backend is allowed to do. These are more important than simply listing CRUD operations.

Business Rules

ID	Rule	Enforced By
BR-01	[e.g. Only verified volunteers may be assigned]	[Middleware / service / DB]
BR-02	[e.g. Closed requests cannot be modified]	[...]
BR-03	[...]	[...]
BR-04	[...]	[...]

State / Workflow Transitions

Use a state diagram for important entities with workflow states.

[INSERT STATE TRANSITION DIAGRAMS HERE]

6. API Design

List all important endpoints first. Then provide detailed contracts for critical endpoints.

API Summary

ID	Method	Endpoint	Auth	Role	Purpose
API-01	POST	/api/...	Required	[Role]	[Purpose]
API-02	GET	/api/...	Required	[Role]	[Purpose]
API-03	PATCH	/api/...	Required	[Role]	[Purpose]
API-04	DELETE	/api/...	Required	[Role]	[Purpose]

Detailed Endpoint Contract

API-01 — [METHOD /api/path]

Field	Details
Purpose	[What does this endpoint do?]

Authentication	[Required / Public]
Authorization	[Allowed roles + resource restriction]
Parameters	[Path / query parameters]
Validation	[Important validation rules]
Success status	[200 / 201 / 204]
Error statuses	[400 / 401 / 403 / 404 / 409 / 500]

Request example

```
{
  "[field]": "[value]"
}
```

Response example

```
{
  "success": true,
  "data": {
    "[field]": "[value]"
  }
}
```

API-02 — [METHOD /api/path]

Field	Details
Purpose	[What does this endpoint do?]
Authentication	[Required / Public]
Authorization	[Allowed roles + resource restriction]
Parameters	[Path / query parameters]
Validation	[Important validation rules]
Success status	[200 / 201 / 204]
Error statuses	[400 / 401 / 403 / 404 / 409 / 500]

Request example

```
{
  "[field]": "[value]"
}
```

Response example

```
{
  "success": true,
  "data": {
    "[field]": "[value]"
  }
}
```

7. Authentication & Authorization

Authentication

Item	Implementation
Authentication mechanism	[JWT / session / OAuth / other]
Credential storage	[Password hashing method / other]
Token / session lifetime	[...]
Refresh mechanism	[... / Not used]

RBAC Matrix

Use one matrix for the whole backend. It should match the authorization logic implemented in the backend.

Action	[Role 1]	[Role 2]	[Role 3]	[Role 4]
Create [resource]	✓ / X	✓ / X	✓ / X	✓ / X
View [resource]	✓ / X	✓ / X	✓ / X	✓ / X
Update [resource]	✓ / X	✓ / X	✓ / X	✓ / X
Delete / close [resource]	✓ / X	✓ / X	✓ / X	✓ / X

Security Controls

Control	Approach
Input validation	[Library / method]
Authorization	[Middleware / guard / service checks]
Password security	[Hashing approach]
Sensitive data	[How secrets / sensitive fields are handled]
Rate limiting	[If applicable]

8. Background Processing & Services

If none are used

Write "None". Do not invent a queue, worker, cron, or event system just for the template.

ID	Process / Service	Trigger	Purpose	Technology	Failure Handling
BG-01	[Cron / Queue / Worker]	[Trigger]	[Purpose]	[Tech]	[Retry / log / fallback]
BG-02	[...]	[...]	[...]	[...]	[...]

9. External Integrations

If none are used

Write "None".

ID	Service	Purpose	Direction	Data exchanged	Failure Behaviour
EXT-01	[Service]	[Purpose]	Backend → Service	[What is sent/received]	[What happens if unavailable]
EXT-02	[...]	[...]	[...]	[...]	[...]

10. Error Handling & Reliability

Standard Error Response

```
{
  "success": false,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "Resource does not exist"
  }
}
```

HTTP Status Conventions

Situation	Status	Example
Invalid input	400	[Missing / invalid field]
Unauthenticated	401	[Missing / invalid credentials]
Unauthorized	403	[Role/resource restriction]
Not found	404	[Resource does not exist]
Conflict	409	[Duplicate / invalid state transition]
Server failure	500	[Unexpected backend error]

Important Failure Scenarios

ID	Failure	Expected Backend Behaviour
ERR-01	[Database failure]	[...]
ERR-02	[External service unavailable]	[...]
ERR-03	[Invalid state transition]	[...]
ERR-04	[Duplicate request / operation]	[...]

11. Testing Strategy

List the critical backend behaviours that should be verified during implementation. Focus on requirements, authorization, workflows, and edge cases.

Test ID	Requirement / Flow	Scenario	Expected Result
T-01	REQ-01	[Valid request]	[Expected response / DB change]
T-02	REQ-01	[Invalid input]	[Expected error]
T-03	REQ-02	[Unauthorized role]	[403 / rejection]
T-04	REQ-03	[Invalid state transition]	[Rejected]
T-05	REQ-04	[Critical happy path]	[Expected result]

Critical Test Coverage

- Authentication / unauthenticated access
- Role-based authorization
- Resource ownership / assignment restrictions
- Validation and malformed input
- Important state transitions
- Critical database relationships and constraints
- External-service or background-process failure, where applicable